

ML System Design in a Hurry

Evaluation 🎵

Evaluations for important patterns of ML system designs, built by FAANG managers and staff engineers.

Every ML system design problem will require some sort of evaluation. Evaluations are part of the optimization loop, and showcasing your ability to evaluate (and by proxy, improve) ML systems is a key part of the interview process. The hard part is that most engineers aren't working on production systems of every variant: so if you've been working on time series forecasting and are asked in an interview about how classification systems are evaluated, you're at a disadvantage.

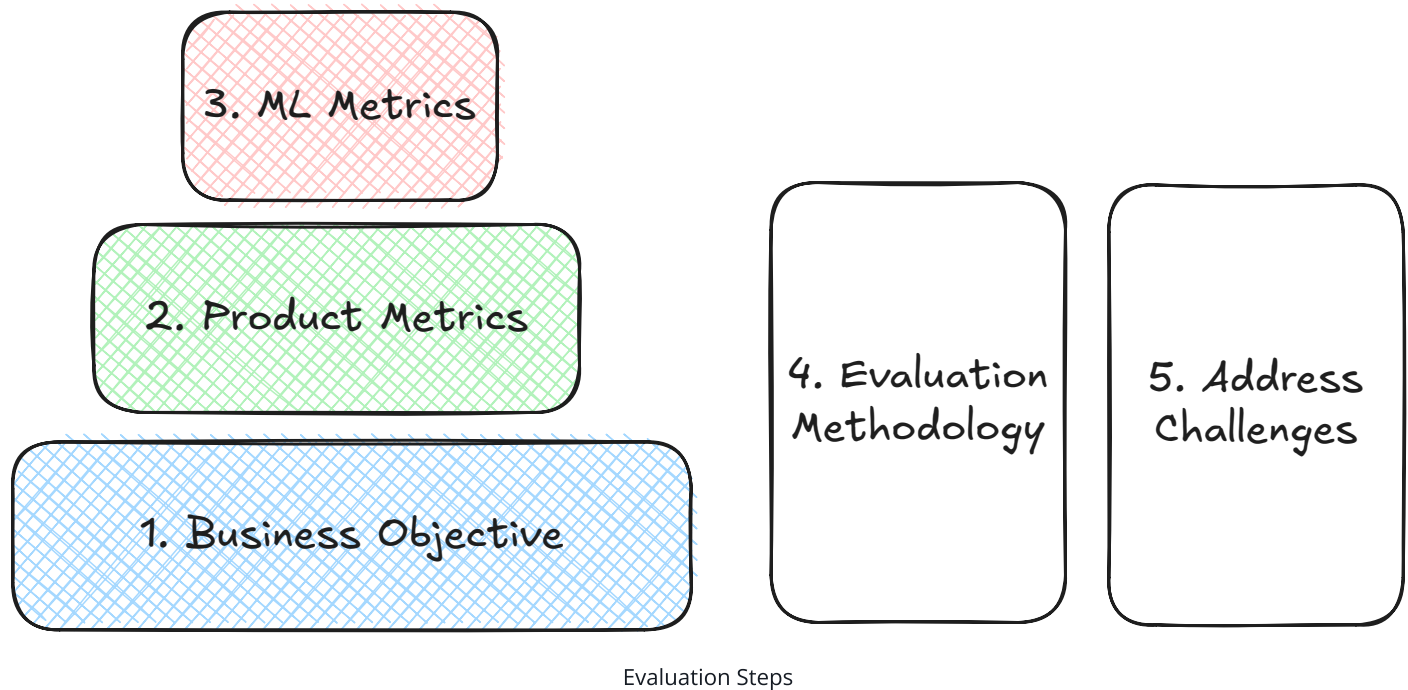
In this guide, we're going first walk through a general evaluation framework which you can use for any ML system design problem. Then we'll talk through the evaluation challenges for popular types of ML systems seen in interviews and discuss metrics and techniques you can use for them. The goal here is to establish breadth necessary to be ready for most interview questions, with references to build up your depth.

General Evaluation Framework for Interviews

Most production ML systems are not trivial to evaluate. There may be subjectivity at play, feedback loops may be long, and there may be many pieces of the system working in concert. As such, when you approach an evaluation challenge you'll want to be structured in how you approach it. We recommend this structure:

1. **Business Objective:** We'll start with the business objective. If you're following our [delivery framework](#), the business objective will be top of mind. Working backwards from this objective will ensure your metric is tethered to something real and valuable, not an arbitrary vanity evaluation.
2. **Product Metrics:** Next we'll work up to the product metrics: explain which user-facing metrics will indicate success. Think about what you can measure in the performance of the product or system which will indicate success.
3. **ML metrics:** With product metrics in place, we can detail technical metrics that align with the product goals. These should be metrics you can measure in the performance of the ML system, often without requiring new inputs (labels, user feedback, simulation results, etc.)

4. **Evaluation Methodology:** With our metrics in place, we'll discuss how we can measure them. Outline both online and offline evaluation approaches. Oftentimes your offline evaluations will be a proxy for the online evaluation, but geared towards rapid iteration.
5. **Address Challenges:** Finally, your evaluation will invariably include some challenges. Imbalanced data, labelling costs, fairness issues, etc. You'll finalize the evaluation discussion by discussing potential pitfalls and how you'd mitigate them.



We'll talk through each of these steps with examples below. Remember that the work of an ML engineer is primarily focused on facilitating an optimization: demonstrating your ability to show you can work at each layer of the evaluation "stack" will show your interviewer you're ready to optimize real production systems.

Classification Systems

In classification systems, we're taking some input (e.g. an image and some text) and we're predicting some output (e.g. a classification label). Classification systems will often have a labelling component with non-trivial cost, so an important consideration is how resource-efficient you can be in conducting these evaluations. They also frequently "fill in" for humans doing manual work, which provides good baselines for thresholds and clues to the right metrics to use to evaluate.

Example problems: Content moderation, spam detection, fraud detection

Business Objective

For classification systems, the business objective will often be downstream of the classification. Ask yourself "what action is taken as a result of the classification?" and "how does that action impact the business?" For example, in a content moderation system, the primary business goal is minimizing harmful content exposure while avoiding false positives that might frustrate legitimate users. This impacts user retention and operational review costs.

Product Metrics

Product metrics will vary wildly by application, but some common ones include:

- User retention rate
- Time to label (moderate/evaluate/judge/etc)
- User satisfaction scores
- Operational review costs
- Appeal rate for moderation decisions
- Downstream costs associated with errors (both false-positives and false-negatives)

ML Metrics

For measuring classification systems, it's common to use the language of precision and recall.

Precision is the percentage of positive predictions that are actually positive. **Recall** is the percentage of actual positives that the model correctly identifies. Classification performance can often be summarized with a precision-recall curve, which shows the trade-off between precision and recall for different threshold values.



We can tune the threshold arbitrarily according to the business objective: if we care more about precision, or more about recall, we can increase or decrease the threshold. For many applications, fixing precision at some arbitrary threshold — often around human-level performance — is a good way to ensure the system is useful.

Some common metrics include:

- **Precision** at your operating threshold (e.g., 95%)

- Recall at that precision
- ROC-AUC for binary classification
- F1 score (harmonic mean of precision and recall)
- PR-AUC for overall model quality
- False positive rate for different content segments
- Per-class performance for multi-class classification systems



If your problem has strong class imbalance (e.g. 99% of the data is negative, 1% is positive), PR-AUC is going to be a far better metric than ROC-AUC. ROC-AUC can look fantastic even if the classifier is completely useless for highly imbalanced problems.

Evaluation Methodology

For online evaluation, you'll often want to implement a shadow mode test where the model makes predictions but doesn't take action, allowing reviewers to validate results. Once confidence is established, move to an A/B test measuring both technical metrics and business impacts like user retention and reviewer workload.

For offline evaluation, establish a balanced test set with stratified sampling to ensure adequate representation of all content categories. Use precision-recall curves to find the optimal threshold that maintains your required precision while maximizing recall. The important thing is that your offline evaluation is correlated with the outcomes of your online evaluations!

Address Challenges

Classification systems face several significant evaluation challenges that need to be addressed for effective deployment:

Class Imbalance

Most real-world classification problems have highly imbalanced class distributions. For example, in fraud detection, legitimate transactions vastly outnumber fraudulent ones. This affects the datasets you assemble for evaluation, and the metrics you choose to evaluate. For highly imbalanced problems, PR-AUC is a far better metric than ROC-AUC. In interviews, you

may also want to talk about a loop for discovering previously unseen instances of the minority class.

Label Efficiency

Obtaining high-quality labels is often expensive and time-consuming, especially for specialized domains requiring expert knowledge. Especially in the presence of class imbalance, random sampling is a poor way to acquire labels which will reduce variance in your evaluation metrics. Employing stratified sampling approaches using classifiers scores can be useful, or even using active learning to prioritize labeling the most informative examples.

Estimating Prevalence

In many applications, the true prevalence of positive cases in production may be hard to measure: in most cases you won't know the "right" answer without expending considerable resources (e.g. running a human review), hence the reason for the ML system in the first place. Random sampling is a good, unbiased way to estimate prevalence but suffers from high variance. Imagine you have a problem where $<1\%$ of the data is positive: if you randomly sample 100 examples you'll only have 1 positive example to estimate prevalence from.

Feedback Loops

Finally, classification systems can create feedback loops where the model's predictions influence future test data, potentially amplifying biases or errors over time. Some ways to solve this include: regularly inject randomness into the system (e.g. withhold actions you might otherwise take) to explore the full data space, or maintain a golden set of examples unaffected by the model's decisions.

Recommender Systems

In recommender systems, we present a ranked list of items (movies, products, news, users to follow) that we believe a user will engage with. Unlike binary classification, the system's value is tied to *ordering* and *diversity* rather than a single yes/no prediction. Evaluation is therefore less about "Did we get the label right?" and more about "Did we surface the *right* set of items, in the

right order, at the *right* time—while staying within business constraints such as inventory, policy, or contractual obligations?"

Example problems: Product recommendations on an e-commerce site, video recommendations on a streaming platform, friend suggestions in a social network.

Business Objective

The action triggered by a recommender is a user decision: click, purchase, watch, or follow. That decision translates into revenue (direct sales, ad impressions, subscription retention) and also into *exposure cost*: surfacing the wrong item can waste scarce inventory slots or even drive users away. Always ask yourself:

What is the dollar value of one additional relevant recommendation?

What is the opportunity cost of showing an irrelevant or policy-violating item?

For instance, a streaming service may prize *completion rate* (finished episodes boost retention), while a retail marketplace may focus on *gross merchandise value* net of return risk.

Product Metrics

Because recommendation funnels are long, product metrics usually accumulate over sessions rather than single impressions. These make for more interesting, longitudinal evaluation challenges:

- **Session watch/purchase rate** per user
- **Average revenue per user (ARPU)** or **Gross Merchandise Value (GMV)/user**
- **Retention or churn-deferral rate** over N-day windows
- **Inventory utilization** (how evenly the catalog is surfaced)
- **Dwell time** (are we creating "doom-scrolling" that later harms satisfaction?)



Optimizing short-term results can often cannibalize long-term results. Interviewers love to push on this.

ML Metrics

Classic information retrieval metrics dominate, but each carries pitfalls:

- **Mean Reciprocal Rank (MRR)**: sensitive to the very first relevant item—great for "top pick" use-cases.
- **Normalized Discounted Cumulative Gain (NDCG)**: discounts relevance by log-rank; good all-rounder.
- **Hit@K / Recall@K**: fraction of sessions where at least one relevant item appears in the top K.
- **Coverage**: proportion of catalog shown over a time window—critical for cold-start sellers and long-tail content.
- **Calibration / Expected Rating Error**: does the score distribution match observed engagement probabilities?



Offline ranking metrics only approximate user utility. This makes online metrics and repeated validation of the correlation between offline and online metrics a must for recommender systems.

Evaluation Methodology

Offline: Build a *leave-one-interaction-out* test set so each user appears in train and test but with different timestamps. Replay the candidate-generation + ranking pipeline to compute ranking metrics. Watch for temporal leakage: including tomorrow's interactions in today's training will yield spectacular but meaningless scores.

Online: Deploy in *shadow-rank* mode—your model reorders items but the served order still comes from the baseline. Compare click-through on items that would change position. Once safe, graduate to an A/B bucket. Measure at least one short-term (CTR) and one long-term (retention) metric; if they diverge, you've found a trap. Remember that your experiments must be long enough to capture long-term effects!



Interleaving tests can be used to detect ranker superiority with fewer users than a full A/B. The idea is pretty simple: A/B tests assign different users to ranker A or ranker B. That

produces an unpaired comparison: every metric still contains user-level noise, so it takes a lot of traffic (often millions of impressions) to detect small lifts. Interleaving instead shows one **mixed** list to the same user, turning the comparison into a paired test. Because every click is simultaneously evidence for one ranker and against the other, variance drops sharply: typically 10-20x less traffic than an A/B for the same power.

See [Paired Difference Tests](#) for more insight here.

Address Challenges

Evaluation Horizon

Immediate clicks are easy to measure; the true goal (renewed subscription six months later) is not. Use proxy metrics plus periodic hold-out cohorts to estimate long-range effects. Techniques like **counterfactual evaluation with importance weighting** help, but high variance is a constant menace.

Feedback Loops

Just as in classification, recommender outputs influence future training logs. Over time the model trains on its own echo chamber. Solutions: periodically inject exploration traffic (e.g., ϵ -greedy or Thompson Sampling), maintain a uniformly-sampled "golden" interaction set, and retrain with **counterfactual logging** to debias.



A/B tests can fail silently if the treatment model collects data that the control model never sees. Always plan for *replayability*: log all candidates and features, not just the ranked list.

By articulating these objectives, metrics, and pitfalls, you'll demonstrate a holistic grasp of recommender-system evaluation—precisely what interviewers look for in a system-design setting.

Search & Information Retrieval Systems

Search and information-retrieval (IR) systems take a query (text, voice, image, etc.) and return an ordered list of results. Unlike binary classifiers, IR systems must **rank** thousands of candidates and satisfy users, usually in a very short time window (e.g. 100ms), so evaluation balances *relevance*, *speed*, and *business impact*.

Example problems: Web search, e-commerce product search, code search, enterprise document retrieval

Business Objective

Ask "what happens when the user finds (or fails to find) the right result?" and "how does that affect the business?"

- In web search, satisfied users return tomorrow, driving ad revenue.
- In product search, higher relevance lifts conversion and average order value.
- In enterprise search, faster retrieval lowers employee time-to-answer and support costs.



Clicks alone are usually a poor proxy for user satisfaction. You'll need to think about other metrics which will demonstrate the system provides value, like observing bounce rates from the resulting clicks.

Product Metrics

Typical top-line signals include:

- Query success rate (did the session end with a click/purchase?)
- Click-through rate (CTR) on the first results page
- Time to first meaningful interaction
- Session abandon rate
- Revenue or conversions per search
- Latency-p99 for search requests
- Query reformulation rate (proxy for dissatisfaction)

ML Metrics

IR quality is usually summarized with rank-aware metrics calculated at cut-off k . k is fixed as a product parameter: if your UI shows 10 results above the fold, optimizing **NDCG@k** aligns well with user satisfaction.

- **Precision@k / Recall@k** – fraction of the top k results that are relevant vs. fraction of relevant docs retrieved
- **Mean Reciprocal Rank (MRR)** – focuses on the first relevant result
- **Normalized Discounted Cumulative Gain (NDCG@k)** – weights high-rank relevance more heavily
- **Mean Average Precision (MAP)** – averages precision across recall levels
- **Hit Rate / Success@k** – at least one relevant doc in top k



Click logs are biased by previous rankings ("presentation bias"). If you build labels from clicks, you'll need debiasing techniques like inverse-propensity weighting or deterministic interleaving. Many interviewers like to probe this point!

Evaluation Methodology

Offline — Build a held-out set of (query, doc, graded-relevance) triples. Compute NDCG@k, MRR, latency, and cost. Maintain diversity: head, torso, and long-tail queries; freshness-sensitive vs. evergreen; different locales.

Online — Deploy in *shadow* (rank but don't serve) to check latency & safety. Graduate to A/B or interleaving tests measuring CTR, revenue, latency, and downstream actions (purchases, page views). Verify offline gains correlate with online wins; if not, revisit labeling or bias correction.

Address Challenges

Query Ambiguity

Many queries have multiple possible interpretations or intents (e.g., "jaguar" could refer to the animal, car, or sports team). This ambiguity makes it difficult to evaluate relevance since the "correct" results depend on user intent. To address this challenge, you can implement intent classification systems that detect and categorize query intent, allowing for per-intent evaluation

of results. Diversification strategies ensure results cover multiple possible intents in proportion to their popularity. User behavior analysis through click patterns helps understand intent distribution, while tracking query refinements provides insight into how users naturally disambiguate their queries.

Long-Tail & Sparse Judgments

The vast majority of search queries are unique or very rare (the "long tail"), making it impractical to collect relevance judgments for every possible query. Active learning approaches help prioritize which queries to label by selecting those that will most improve model understanding. Query clustering techniques allow sharing of relevance judgments among similar queries, while synthetic query generation creates artificial queries to test specific ranking aspects. Transfer learning enables applying relevance signals from head queries to similar tail queries. Modern approaches also leverage large language models for zero-shot evaluation of relevance without explicit labels.

Freshness & Recency

Search systems must balance serving fresh content with maintaining result quality, especially for time-sensitive queries. This requires tracking temporal relevance to understand how quickly results become stale for different query types. Systems need to monitor crawl latency from content creation to searchability and maintain metrics on index freshness. Query classification helps identify which queries require fresh results versus those that can use evergreen content. Time-based decay functions can be applied to relevance scores to naturally deprecate older content when appropriate.

Feedback Loops

Search systems can create self-reinforcing cycles where popular results get more clicks, leading to higher rankings and more clicks. Breaking this cycle requires regularly injecting randomness into rankings to gather unbiased feedback. Position bias can be addressed through inverse propensity scoring and click data debiasing. Interleaving techniques enable system comparison through paired tests with reduced variance. Maintaining golden sets unaffected by feedback loops provides stable evaluation baselines, while diversity metrics ensure coverage of the full result space rather than just popular items.

Generative AI Systems

In generative systems, the model produces new content like text, images, code, audio rather than selecting from predefined labels. The biggest evaluation hurdle is that "correctness" is often subjective: a response may be valid in multiple ways, yet still miss brand tone, factual accuracy, or user intent. Resource-wise, reference answers are costly to gather, and human review is slow, so smart sampling and proxy metrics become essential.

Example problems: Chat assistants, code generation, image synthesis, customer support bot

Business Objective

Ask "What *value* does the generated content unlock?" and "What *harms* must we prevent?"

A support chatbot's goal is deflecting tickets without frustrating customers; an image tool aims to boost ad-click-through while avoiding brand-unsafe outputs. These objectives translate into retention, revenue lift, and moderation workload.

Product Metrics

Product metrics for generative AI systems are *highly* dependent on the application. Here are some common ones:

- Task success rate (e.g., ticket fully resolved)
- Average handle time (when humans intervene)
- User satisfaction / Net Promoter Score (NPS, "how likely are you to recommend this product to a friend?")
- Brand-safety incident rate & review cost
- Prompt-response latency
- Down-stream engagement (clicks, watch-time, etc.)

ML Metrics

Generative quality rarely fits a single scalar, and interviewers aren't expecting just a single metric. Use a *portfolio* of metrics and don't shy away from setting up custom evaluators:

- **Automated overlap scores:** BLEU, ROUGE, METEOR (cheap but brittle)
- **Semantic similarity:** BERTScore, BLEURT
- **Factuality / hallucination rate:** task-specific fact checkers
- **Toxicity & bias scores:** perspective or hate-speech models
- **Diversity metrics:** self-BLEU, distinct-n
- **Human ratings:** pairwise preference or Likert scale
- **Custom metrics:** % of refusals, % of hallucinations, etc.



Evaluating generative systems is an ongoing process. Combine automated filters (quick, high-recall) with periodic human evaluation (slow, high-precision). Make sure you have a way to track correlation so the cheap signal stays honest. And be ready for saturation, you'll need to be evolving your metrics as your system matures.

Evaluation Methodology

Offline:

Create a stratified test set of prompts covering intents, languages, edge-cases, and policy red-lines. For text, collect multiple reference answers or use pairwise ranking. Measure quality metrics, run toxicity detectors, and slice by domain.

Online:

Ship in (shadow) mode first: generate suggestions but hide them from users, logging quality signals. Graduate to A/B where the new model handles a % of traffic; monitor business KPIs + safety dashboards. Always keep a "golden canary" set of prompts served by a legacy model for drift detection.

Address Challenges

Evaluating generative AI systems is the grand challenge of the coming decade, so there are no easy answers here but there are a lot of potholes to avoid. Interviewers are looking to see that

you can think through these challenges, or in some cases (for a generative AI-focused loop), that you've seen them yourself.

Subjective Quality

Unlike classification tasks, generative outputs often have no single "correct" answer, making evaluation inherently subjective. To address this, implement multi-reference evaluation by collecting multiple valid outputs for each input. Build scalable workflows for expert review through human evaluation pipelines, and train preference learning models to predict human preferences. Breaking down evaluation into specific aspects like fluency, coherence, and style helps make the subjective more measurable. Regular re-evaluation of hard cases and edge cases ensures the system maintains quality across the full range of outputs.

Hallucination & Factual Consistency

Generative models can produce confident but incorrect statements, requiring careful evaluation of factual accuracy. Implement source attribution systems to track which generated content can be verified against source material. Automated fact checking against knowledge bases helps catch obvious errors, while retrieval augmentation improves accuracy by grounding generations in verified information. Monitor the correlation between model confidence and correctness through calibration metrics, and develop taxonomies to classify different types of hallucinations based on their severity and impact.



Hallucination severity explodes with rare or niche prompts—exactly the queries reviewers won't see often. Target them deliberately in test sets.

Safety And Policy Compliance

Toxic, biased, or illicit outputs carry legal and brand risk. The evaluation of safety systems must focus particularly on false-negative rates, as missing a problematic output often carries more risk than incorrectly flagging a safe one. Regular red-team testing and adversarial evaluation help identify potential vulnerabilities.

Evaluation Cost

Human evaluation is expensive and slow, requiring efficient use of review resources. Active learning approaches help prioritize review of uncertain or novel outputs, while automated filters provide fast initial screening. Develop efficient sampling strategies that cover the input space effectively while minimizing redundant review. Create proxy metrics that correlate well with human judgment to reduce reliance on manual review. Optimize the distribution of review resources across different quality dimensions based on their impact and risk. Stratified sampling is a good way to ensure you're covering the full range of inputs.

Distribution Shift

User behavior and content patterns evolve over time, requiring continuous evaluation adaptation. Implement drift detection systems to monitor changes in input distribution and model performance. Maintain rolling windows of recent traffic in test sets to ensure evaluation remains relevant (i.e. hold out based on time, not just randomly). Use stable benchmark sets as canaries to detect model degradation, and develop clear adaptation strategies for regular model updates. Version control systems should track changes in both model behavior and evaluation criteria to maintain consistency over time.

Appendix: Intuitive Explanation of Key Metrics

Classification Metrics

Precision: The percentage of positive predictions that are actually positive.

- Intuition: "When the model says something is harmful content, how often is it right?"
- Calculation: $(\text{True Positives}) / (\text{True Positives} + \text{False Positives})$

Recall: The percentage of actual positives that the model correctly identifies.

- Intuition: "What percentage of all harmful content does the model catch?"
- Calculation: $(\text{True Positives}) / (\text{True Positives} + \text{False Negatives})$

PR-AUC: Area under the Precision-Recall curve.

- Intuition: "How well does the model balance precision and recall across different thresholds?"
- Ranges from 0 to 1, with higher values being better

F1 Score: The harmonic mean of precision and recall.

- Intuition: "A single number balancing how many positives we catch vs. how accurate our positive predictions are"
- Calculation: $2 * (\text{Precision} * \text{Recall}) / (\text{Precision} + \text{Recall})$

ROC-AUC: Area under the Receiver Operating Characteristic curve.

- Intuition: "How well does the model distinguish between classes across different thresholds?"
- Ranges from 0 to 1, with higher values being better
- Note: Can be misleading for imbalanced datasets

Ranking Metrics

NDCG (Normalized Discounted Cumulative Gain):

- Intuition: "How well are the most relevant items ranked at the top?"
- Calculation: Sum the relevance scores of results, discounted by position (items lower in the ranking contribute less), then normalize by the "ideal" ranking
- Higher positions matter much more than lower ones

MAP (Mean Average Precision):

- Intuition: "Average of precision values calculated at each relevant item in the ranking"
- Rewards rankings where relevant items appear earlier
- Calculation: For each query, find the average precision at each relevant result, then average across all queries

MRR (Mean Reciprocal Rank):

- Intuition: "The average of $1/\text{position}$ for the first relevant result"
- Focuses solely on the position of the first relevant item
- Perfect score (1.0) means the first result is always relevant

Image Generation Metrics

FID (Fréchet Inception Distance):

- Intuition: "How similar is the distribution of generated images to real images?"
- Lower scores mean the generated images have similar statistical properties to real images
- Measures both quality and diversity

CLIP Score:

- Intuition: "How well does the generated image match the text prompt?"
- Higher scores indicate better text-image alignment
- Based on how close the image and text embeddings are in a shared space

Text Generation Metrics

Perplexity:

- Intuition: "How surprised is the model by the actual text?"
- Lower values indicate the model assigns higher probability to the correct tokens
- Calculation: $2^{(\text{negative log likelihood per token})}$

BLEU/ROUGE/BERTScore:

- Intuition: "How similar is the generated text to reference text?"
- Measure overlap of n-grams (BLEU/ROUGE) or semantic similarity (BERTScore)
- Higher values indicate greater similarity to references

Remember, no single metric tells the whole story. The best evaluation approaches use multiple complementary metrics and align them with business objectives.

Test Your Knowledge

Take a quick 15 question quiz to test what you've learned and identify gaps.

 [Start Quiz](#)